

Resumen del capítulo: Resolver tareas relacionadas con el machine learning

Descripción del ejercicio

Formular una tarea de negocio

Cuando formulamos una tarea, las preguntas principales a responder son:

- ¿Qué estamos pronosticando y qué tarea de negocio puede resolver esto?
- ¿Qué datos tenemos?
- ¿Quién va a utilizar nuestro modelo y cómo (cómo se integra en los procesos de negocio)?
- ¿Qué resultados esperamos alcanzar?
 - ¿Cómo se ha resuelto el problema hasta ahora? ¿El método fue exitoso?
 - ¿Qué efecto podría tener tu modelo en el negocio?
 - ¿Tienes suficientes recursos (tiempo, personal, finanzas, recursos computacionales)?
 - ¿Qué métricas utilizarás para evaluar el desempeño de tu modelo?
 - ¿Existe algún referente en el mercado para la aplicación del aprendizaje automático para tareas similares?

Definir la tarea y traducirla al lenguaje del aprendizaje automático

Si eres capaz de dar respuestas claras a todas esas preguntas y confirmar que realmente necesitas utilizar el aprendizaje automático, entonces traduce el enunciado de negocio en lenguaje de ML (*machine learning*). Esto implica identificar:

- El tipo de tarea (aprendizaje supervisado o no supervisado; clasificación, regresión o *clustering*).

- Características:
 - ¿Qué datos seleccionar como características?
 - ¿Cuál es el tamaño de la muestra?
 - ¿Cuál es el periodo?
 - ¿Qué tan buenos son los datos?
 - ¿Los datos tienen una estructura temporal?
- Métricas que utilizarás para optimizar y evaluar tu modelo.
- Si el modelo tiene limitaciones de:
 - velocidad;
 - interpretabilidad del resultado;
 - exactitud;
 - tiempo necesitado para el desarrollo.
- Los algoritmos que vas a utilizar (según los puntos anteriores).

EDA: Análisis de la calidad de las características

En la primera etapa defines la tarea y los datos que vas a utilizar; después, de forma crucial, obtienes los datos en sí (probablemente como un archivo CSV); en la siguiente etapa, los estudias. Inmediatamente se te pueden ocurrir numerosas hipótesis sobre el proceso que estás modelando o sobre sus características más importantes.

En el primer capítulo describimos la esencia del **análisis exploratorio de datos** (EDA) que tendrás que llevar a cabo.

Los objetivos de este tipo de exploración son:

1. Evaluar la calidad de los datos y el volumen de preprocesamiento requerido.
2. Examinar las distribuciones y las correlaciones mutuas y detectar anomalías, si se presentan.
3. Formular hipótesis iniciales de acuerdo con las características y la variable objetivo.

Aquí hay algunas preguntas útiles que necesitas responder cuando evalúes la calidad de los datos:

- ¿De qué tamaño es el *dataset*?
- ¿Qué características incluye? Si son anónimas (lo que sucede con mucha frecuencia en competencias especializadas), es mejor estudiar qué significa cada característica para profundizar en el proceso de negocio.
- ¿Qué tipos de características tienes? Normalmente son **numéricos** o **categoricos**.
- ¿La variable objetivo tiene una estructura de tiempo? (¿Necesitamos predecir series temporales?) Esto determina qué métodos para segmentar datos en sets de entrenamiento y validación necesitarás aplicar en las etapas futuras y qué características derivadas (basadas en las originales) podrás utilizar.
- ¿Cuántos valores ausentes hay? Este es un punto importante. A veces, ves de inmediato que algunas características son realmente útiles pero, desafortunadamente, solo un 1% de las observaciones las incluyen, así que no pueden utilizarse en un modelo. En esta etapa puedes decidir cómo procesar los valores ausentes: ya sea eliminándolos del *dataset* o rellenándolos desde el "pasado" o "futuro".

EDA: formular hipótesis

Una vez que tengas una idea aproximada de las características, puedes echarles un vistazo más de cerca.

- Estudia la distribución de las características numéricas. Traza histogramas para ellas. Esto te permitirá ver si hay algún valor atípico y examinar cómo se comportan las características. Quieres que todo sea más o menos normal.
- Si tus datos tienen una dimensión temporal, traza los gráficos de distribución a lo largo del tiempo. Esto es especialmente importante cuando estás tratando con procesos industriales. Te permitirá ver si el comportamiento de las características

cambia a lo largo del tiempo. Dijimos anteriormente que solo tiene sentido utilizar un modelo cuando tienes la certeza de que el comportamiento de las características no cambiará mucho. De otro modo, el modelo no será útil.

- Utiliza la distribución de las características y las variables objetivo para determinar si hay valores atípicos. Estos resaltan en los histogramas y otros gráficos.
- Calcula una matriz de correlación y traza un mapa de calor basado en ella.

Puedes calcular una matriz de correlación con una sola línea de código utilizando el método `corr()` del DataFrame:

```
cm = df.corr()
```

Ahora la variable `cm` almacena la matriz de correlación. Para presentarla de forma visual, utiliza `heatmap()` de la librería `seaborn`:

```
sns.heatmap(cm, annot = True, square=True)
```

En esta etapa ya puedes buscar:

- Las características que tienen la mayor correlación con la variable objetivo.
- Las características que se correlacionan fuertemente entre sí. Recuerda que la correlación mutua no es deseable para los modelos lineales, así que presta especial atención a esto si eliges dichos modelos.
- Cómo se ven los gráficos pareados (podría ser útil cuando se junta con el paso anterior, ya que la correlación refleja solo las relaciones lineales, mientras que los gráficos pareados pueden mostrar otras interrelaciones):
 - característica-característica;
 - característica-variable objetivo.

Los gráficos emparejados pueden ser representados con `scatterplot()` de la librería `seaborn`:

```
sns.scatterplot(df['Feature 1'], df['Feature 2'])
```

Una vez que hayas pasado algo de tiempo meditando sobre tus gráficos, serás capaz de formular hipótesis preliminares:

- ¿Qué características podrían ser las más valiosas para el modelo basado en las correlaciones?
- ¿Qué otras características útiles podemos generar de acuerdo a las ya existentes?
- En general, ¿qué tan útil puede ser un modelo con datos de una calidad como esta? Si muchas características son ruidosas con altas varianzas, necesitas reconocer esto desde el principio y no esperar que el ML haga un milagro.

Preprocesamiento de datos

Una vez que hayas visto los datos y tengas una idea de las dificultades que presentan (valores atípicos, valores ausentes, características categóricas), sabrás qué preprocesamiento se necesita para aplicar el algoritmo que has elegido para entrenar tu modelo.

El preprocesamiento de datos generalmente consiste en las siguientes etapas:

1. Procesar valores ausentes.
2. Procesar valores atípicos.
3. Convertir variables categóricas.
4. Estandarizar los datos (para modelos lineales o modelos que son sensibles a la distancia, como *clustering*).

A veces, la selección de características también se considera parte del preprocesamiento, pero esto también se puede hacer en etapas posteriores del desarrollo de un modelo.

Echemos un vistazo a cada etapa en orden.

Procesar valores ausentes

Hay varias formas de tratar con valores ausentes:

- Simplemente elimina las observaciones de valores ausentes. Esto es efectivo cuando hay una gran cantidad de datos y no muchos valores ausentes. En dichos

casos, puedes sacrificar unas cuantas observaciones para las que los valores están ausentes. Generalmente, sin embargo, este es un enfoque radical y no debería ser tu opción.

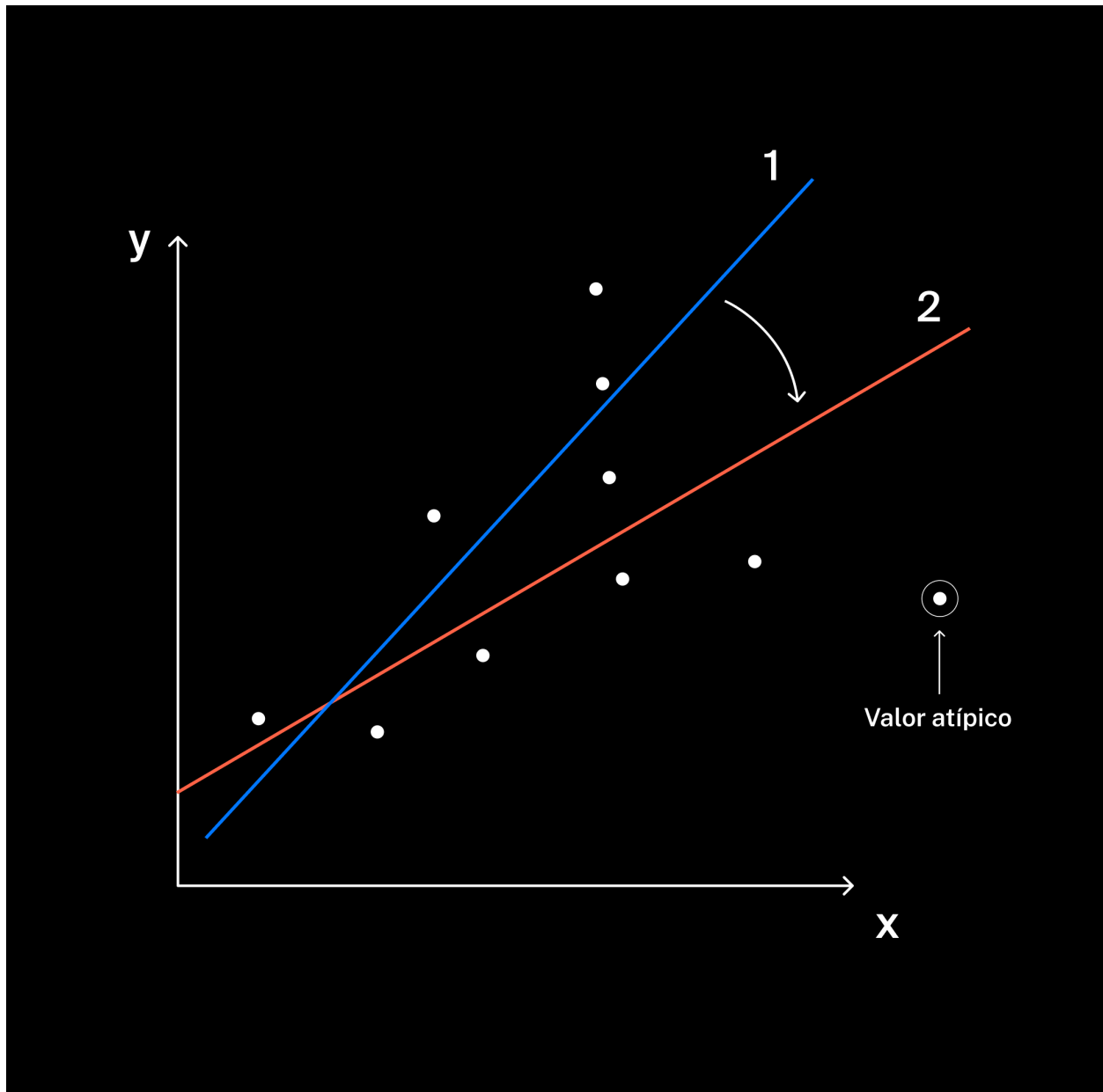
- Reemplázalos con valores vecinos del pasado. Ya conoces el método `fillna()` de la parte del curso de preprocesamiento de datos. Si tus datos están relacionados con el tiempo y uno de los parámetros aparece con menos frecuencia que otros, utiliza `fillna()` con el parámetro `'method'='ffill'`. Aquí tenemos razones para concluir que los valores ausentes deben ser rellenados con los valores del pasado.
- Reemplázalos con valores medios. Este método, al contrario que el de arriba, es típico de las características que no tienen una estructura temporal o nada "a lo que aferrarse". En otras palabras, si el valor de la característica está ausente en una observación, puedes asumir que es la media (no tendrá mucho impacto en la predicción) o la mediana.
- Reemplázalos con valores nulos. Este es otro método radical. Ten especial cuidado si vas a trabajar con algoritmos lineales: los nulos pueden afectar a toda la relación lineal. Si los valores nulos se relacionan con la variable objetivo, este tipo de procesamiento funciona bien para los árboles, ya que, al dividir el conjunto, pueden tener en cuenta el hecho de que una determinada característica no se rellena.
- En algunos casos (particularmente con características categóricas) podemos reemplazar los valores ausentes con el indicador "*unspecified*" (sin especificar). A veces el hecho de que una persona no especifique su ciudad natal en su perfil es importante, y es mejor no reemplazar el espacio en blanco con la respuesta más frecuente.

La estrategia general en cuanto a los valores ausentes es esta: tratar de procesarlos cuidadosamente y asegurarte de que los valores rellenados no causen anomalías cuando estés entrenando el modelo. Si lo hacen, probablemente te deshagas de las características para las que hay muchos valores ausentes.

Procesar valores atípicos

Esta etapa es similar a la anterior en muchos aspectos. La mayoría de los algoritmos son resistentes a los valores atípicos, pero algunos no lo son. Por ejemplo, en un modelo lineal, los valores atípicos pueden tirar de la relación hacia un lado u otro. Mira

cómo solo un punto puede cambiar la relación propuesta por el modelo para esta nube de puntos de la versión 1 a la versión 2:



El procedimiento estándar para trabajar con valores atípicos es el siguiente:

- Define el umbral más allá del cual una observación se considera un valor atípico (por ejemplo, por debajo del 5.º percentil o por encima del 95º).

- Elimina tales observaciones de la muestra (bueno para grandes muestras con un pequeño número de valores atípicos) o reemplázalas con el valor medio o el valor máximo/mínimo para esta característica.

Convertir variables categóricas

Estos son los dos métodos más comunes para transformar variables categóricas:

1. Convertirlas en valores numéricos sin ninguna transformación significativa. Hay dos tipos diferentes de enfoques conceptuales:

- Reemplazar las categorías con valores numéricos (**codificación de etiquetas**, o *"label encoding"*).

Por ejemplo, convirtiendo los valores **string** "Moscú," "Berlín" y "París" en los **numéricos** 0, 1 y 2. Aquí necesitaremos la clase `LabelEncoder()` del módulo `sklearn.preprocessing`:

```
from sklearn.preprocessing import LabelEncoder

print(df['column'].head())

encoder = LabelEncoder() # crear una variable de la clase
df['column'] = encoder.fit_transform(df['column']) # utilizarla

print(df['column'].head())
```

La característica "Ciudad" es ahora numérica en vez de categórica. Pero tiene una importante desventaja: crea relaciones del tipo "mayor/menor" para las categorías. Por ejemplo, para el modelo, Moscú será ahora "menor" que Berlín, ya que su etiqueta numérica es 1 mientras que la de Berlín es 2. Esto no es muy bueno para los modelos basados en la interrelación lineal (por ejemplo: regresión lineal), así que se tienden a utilizar otros enfoques.

- Transformar un campo categórico en un conjunto de binarios (**codificación one-hot**, o *"one-hot encoding"*).

Aquí, en lugar del campo "Ciudad", tenemos los campos "Moscú", "Berlín", "París" y otros, y estas nuevas características tomarán los valores 0 o 1. Para

hacer que esto ocurra, necesitaremos la función `pandas.get_dummies()`. Toma todo el DataFrame como *input* (entrada), identifica las variables categóricas, las transforma en nuevas características (con nombres convenientes) llamadas **variables dummy** y devuelve un DataFrame actualizado:

```
print(df.head())
df = pd.get_dummies(df)
print(df.head())
```

Para hacer tal transformación, también puedes utilizar la clase `OneHotEncoder` del módulo `sklearn.preprocessing`, pero no es tan práctico.

La sustitución binaria de las características categóricas es casi siempre la mejor idea. Pero si hay demasiadas o si una de las características tiene un montón de valores únicos (o ambas), la transformación hará que la matriz (DataFrame) sea demasiado grande. Esto crea problemas para casi todos los modelos de aprendizaje automático, incluso ignorando el hecho de que las matrices grandes significan cálculos lentos. Así que aquí podemos aplicar la codificación de etiquetas o simplemente eliminar características particulares.

2. Crear nuevas características basadas en las existentes. Por ejemplo. si tenemos una característica que consiste en comentarios de texto, podemos utilizarla para generar nuevas características como la longitud del mensaje, la presencia de palabras vacías, el tono positivo o negativo, etc.

4. Estandarizar los datos

En el segundo capítulo aprendiste que muchos algoritmos de ML funcionan mejor con datos estandarizados, donde los valores de característica corresponden a la distribución normal estándar.

La estandarización es absolutamente obligatoria en dos áreas del ML:

- regresión lineal;
- *clustering* y métodos basados en la distancia mutua entre objetos.

La estandarización de una característica determinada implica los siguientes pasos:

- Calcular la media de la característica en la muestra y reducir cada observación por este valor (para centrar los valores alrededor de 0).

- Dividir cada valor resultante entre la desviación estándar (para escalar la dispersión a un valor de 1).

Por consiguiente, dentro del *pipeline* de desarrollo de modelos, la estandarización es la siguiente:

- Segmentar los datos en conjuntos de **entrenamiento y validación**.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

En este ejemplo tenemos una segmentación clásica 80/20.

- Entrenar y aplicar el estandarizador en el conjunto de entrenamiento.

```
scaler = StandardScaler()  
X_train_st = scaler.fit_transform(X_train)
```

Aquí el estandarizador recuerda la media y la desviación estándar de tu conjunto de entrenamiento y, teniendo este conocimiento en cuenta, le aplica la estandarización.

- Aplicar el estandarizador a los **datos de validación**.

```
X_test_st = scaler.transform(X_test)
```

Aquí tú estandarizas los datos de prueba y almacenas la matriz (no el DataFrame) con los valores de característica transformados en la variable `X_test_st`. Pero ten cuidado: estás transformando valores en el conjunto de **validación** basándote en el valor medio y la desviación estándar hallada para el otro conjunto, los datos de entrenamiento.

Esto podría parecer ilógico, y después de tal transformación, la distribución de los datos de **validación** alterados podría seguir sin coincidir con la distribución normal.

Hay dos razones para esto. Primero, cuando pruebas un modelo en la vida real, no conoces la futura distribución de tus valores de característica. Segundo, si las validaciones en el conjunto de validación (y por consiguiente, la desviación estándar y la media) son demasiado diferentes de lo que eran en el conjunto de entrenamiento, tu modelo no funcionará muy bien.

División aleatoria y temporal

Con el preprocesamiento hecho, puedes entrenar el modelo. Tu objetivo no es solo construir un modelo, sino elegir el mejor modelo entre varios. Esto se puede hacer comparando las métricas que consigues de diferentes modelos para los datos de validación. Aquí debemos responder tres preguntas:

1. ¿Cómo dividirás los datos para evaluar el desempeño del modelo con los datos de validación?
2. ¿Qué modelos vas a elegir?
3. ¿Qué métricas utilizarás para juzgar los modelos?

Hay dos enfoques para dividir:

- Aleatoriamente.
- Basándose en el tiempo.

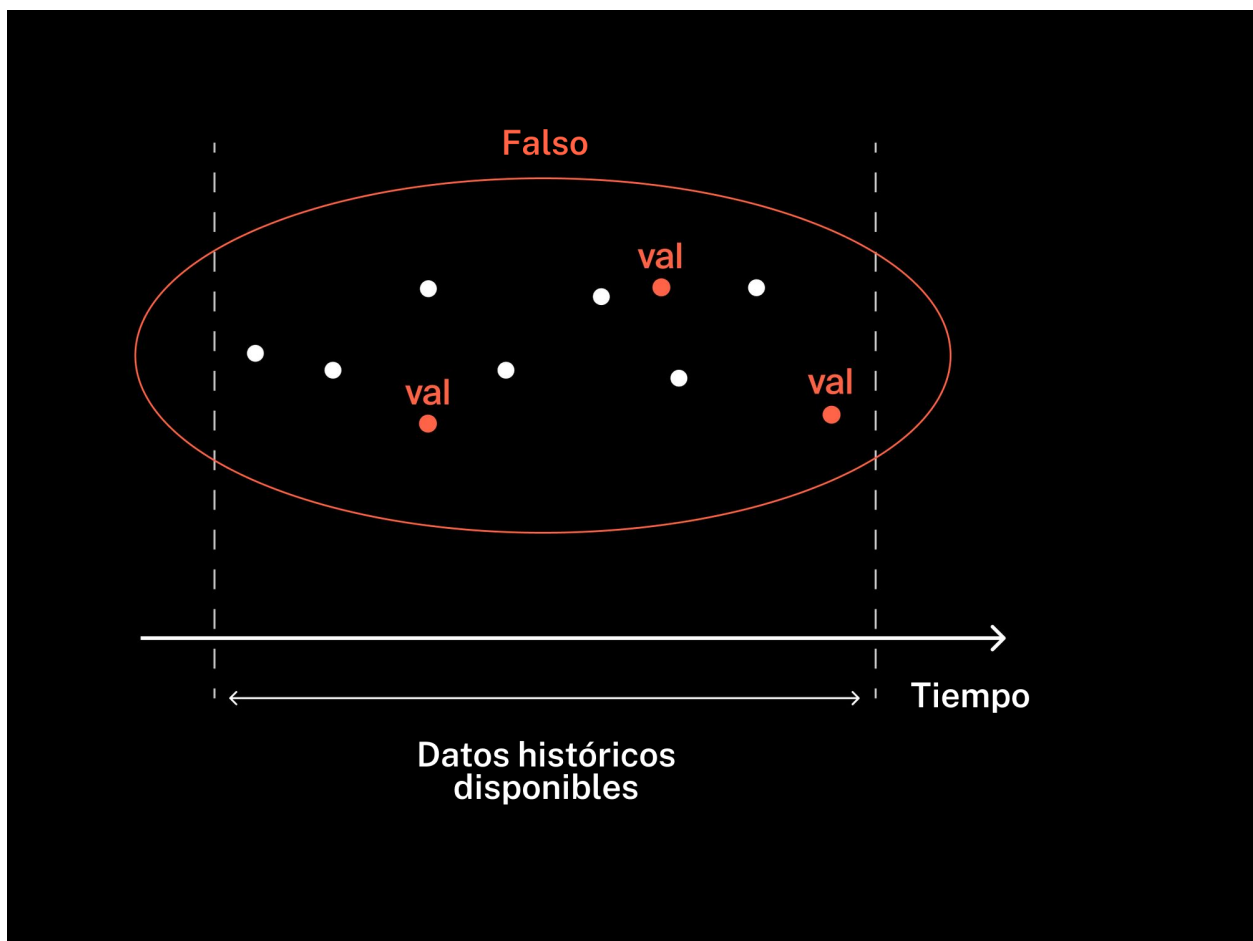
Puedes utilizar una división aleatoria cuando no necesitas predecir una serie temporal. En otras palabras, ignoras el impacto que las observaciones vecinas tienen entre sí.

Toma un enfoque de división temporal cuando predigas el valor de la variable objetivo para observaciones consecutivas. En tales tareas, a menudo predecimos el valor de la variable objetivo "N meses en el futuro" para cada observación: "un mes en el futuro", "dos meses en el futuro", etc. Las características serán los valores actuales de las variables además de otras características basadas en el tiempo, incluyendo aquellas directamente relacionadas con la variable objetivo:

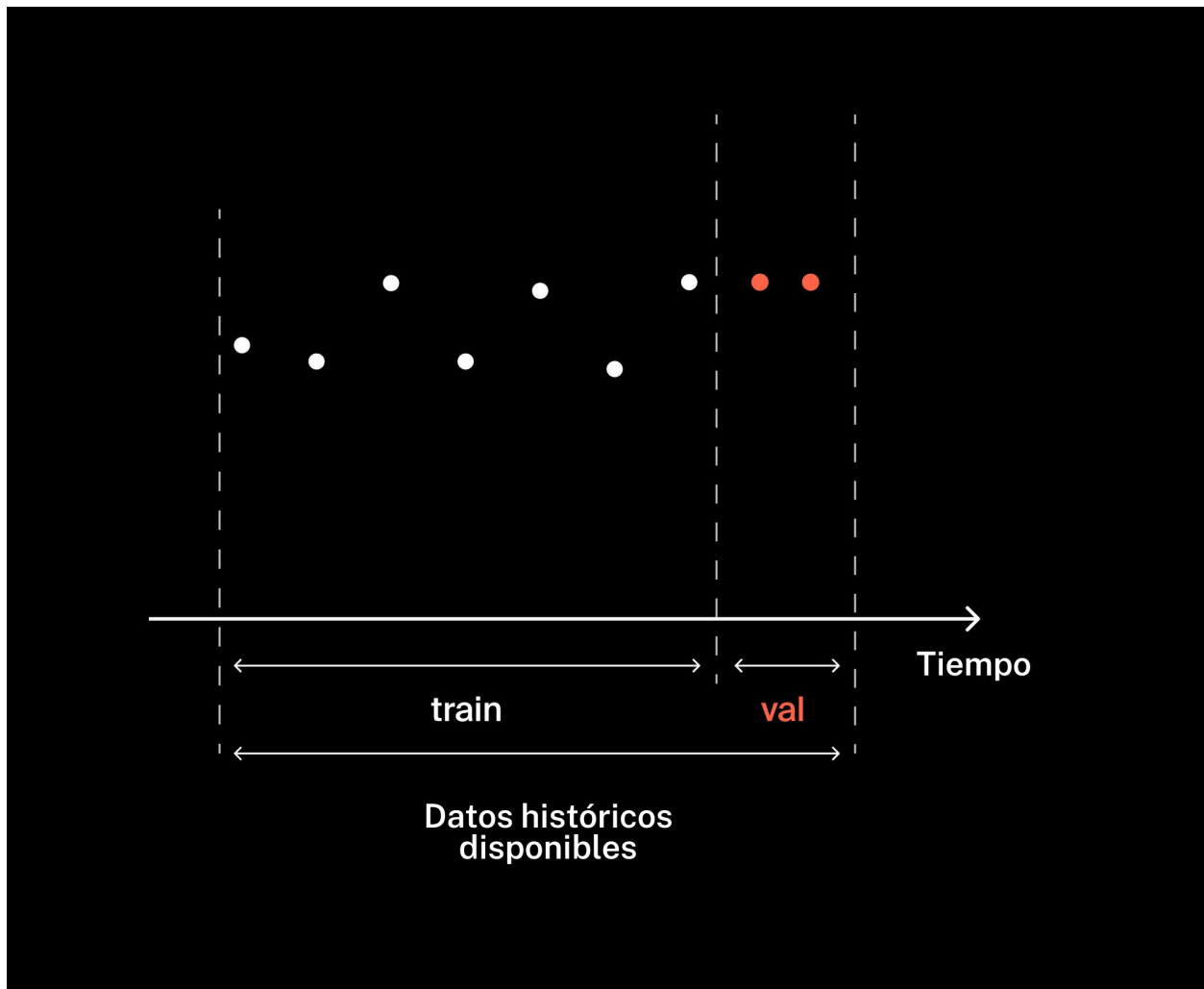
- Valores del pasado (*lags*). Por ejemplo, los valores de las características y la variable objetivo 1, 3, 5, 10, 30 o cualquier otro número de meses atrás (o minutos, dependiendo de la granularidad de los datos, los intervalos de tiempo).

- Los valores de las funciones agregadas (suma, media, desviación estándar, mediana) para las características y la variable objetivo en ventanas de tiempo móviles. Por ejemplo, la suma de los valores de los últimos tres meses, la suma de los valores de los últimos cinco meses, el valor medio de los últimos tres meses o el valor medio de los últimos cinco meses.

Cada observación (punto en el tiempo) se enlaza con la información del pasado, así que aquí la división aleatoria no encajaría. En este caso sería incorrecto utilizar una división aleatoria:



Mejor utilizar una división temporal:



Cuando entrenas usando datos para una serie temporal y la validación para modelar un verdadero conjunto de retención del "futuro" para un algoritmo "del pasado", debes utilizar una **división temporal** (o **basada en el tiempo**). Esto conlleva hacer un conjunto de validación desde el final del periodo histórico disponible en vez de desde puntos en el tiempo seleccionados aleatoriamente. Para este tipo de división la proporción de entrenamiento/validación es 80/20 o 70/30, como en la división aleatoria.

Seleccionar métricas

Para evaluar tus modelos y elegir el mejor, necesitas definir las métricas que describen la esencia de tu tarea.

Hay dos cosas importantes que aún no hemos comentado:

1. Una vez seleccionada la métrica de prioridad, puedes ajustar el proceso de entrenamiento para que optimice esa métrica. Por ejemplo, si estás trabajando en un problema de regresión y utilizas `RandomForestRegressor()`, puedes especificar la métrica directamente cuando optimices el entrenamiento del algoritmo:

```
model = RandomForestRegressor(criterion='mae')
```

Esto hará que tu algoritmo sea menos sensible a los casos en los que el modelo comete grandes errores en observaciones determinadas; es decir, optimizarás el entrenamiento minimizando el valor absoluto del error. Pero si defines el modelo así:

```
model = RandomForestRegressor(criterion='mse')
```

...entonces el modelo sufrirá mayores errores "sutiles", ya que cuadraremos la diferencia entre los valores reales y los predichos, en lugar de simplemente tomar el valor absoluto.

En este caso, estos son los únicos valores posibles del parámetro `criterion`. Otros algoritmos tienen más opciones. A veces, a diferencia de aquí, tu métrica objetivo no estará entre las que pueden ser ajustadas, así que tendrás que encontrar una métrica cercana a la tuya. Imagina que estás resolviendo un problema de clasificación binaria. Has seleccionado la precisión como la métrica básica y esta elección se basa en los objetivos de negocio. El algoritmo se entrena mediante la potenciación del gradiente con `XGBClassifier()` de la librería `XGBoost`. Este algoritmo tiene un parámetro `eval_metric` que te permite ajustar el entrenamiento del modelo para optimizar la métrica elegida. Sin embargo, la precisión no es una de las opciones disponibles. Así que en este caso podemos seleccionar `auc`, que pretende generalmente mejorar tu clasificador ("*classifier*") y luego ajustar la precisión utilizando el umbral.

Por esto es importante leer la descripción del algoritmo en la documentación y averiguar cómo optimizarlo.

2. Para ciertas tareas de negocio tendrás que crear tus propias métricas basadas en las ya existentes para evaluar un modelo. Un ejemplo simple y común es la "tasa

de respuestas falsas". De hecho, esta es la proporción del número de FP (respuestas falsas positivas) para el tamaño de tu muestra. Si cada respuesta te cuesta algo, entonces quieres que tu modelo te dé el menor número posible de respuestas falsas. Al igual que `precision_score`, esta métrica se caracteriza por la precisión de las respuestas y se calcula utilizando este valor:

```
false_positive_rate = 1 - precision_score(y_true, y_pred)
```

La importancia de las características

Has elegido el mejor modelo. Funciona e incluso hace predicciones. Pero para que sean significativas, quienes pretenden utilizar esas predicciones necesitan ser capaces de confiar en ellas. Aún más importante, tú necesitas confiar en ellas. En consecuencia, tienes que entender por qué tu modelo funciona. Aquí la atención cambia hacia la **interpretabilidad** del modelo.

Normalmente, el o la analista prepara una gran cantidad de datos y los manda a la "black box" (caja negra) del algoritmo, que produce una predicción. Pero necesitamos al menos un entendimiento básico no solo de lo que dice el modelo, sino de cómo y por qué lo hace. ¿Qué características se consideraron importantes cuando computó una predicción específica? Esto es crucial para llegar al centro del proceso que se modela y para la evaluación del desempeño del modelo en general. Por consiguiente, necesitamos prestar especial atención a un análisis de la **importancia de la característica**.

Las cosas son más sencillas con modelos lineales. Aquí entrenar supone seleccionar los coeficientes apropiados para cada característica. Dado que estas características son estandarizadas, el valor del coeficiente para cada una de ellas te cuenta su importancia. Digamos que mientras está siendo entrenado, el modelo selecciona los siguientes coeficientes óptimos para la ecuación:

$$y = 132 + 37 * x_1 + 105 * x_2 + 0.3 * x_3 - 79 * x_4$$

¿Qué ocurre si cambiamos el valor de la tercera característica, x_3 , de 0 a 1? El resultado no cambiará considerablemente: un 0.3, lo cual no es mucho. Esto significa que la tercera característica no tiene un fuerte impacto en la predicción del modelo y su

importancia no es tanta. Pero la segunda característica es diferente: incluso si la cambiamos ligeramente (por ejemplo: de 0 a 1), afectará considerablemente al resultado, así que esta característica es muy importante para el modelo. El coeficiente de la cuarta característica es -79. Ya que es negativo, si incrementamos el valor de la característica, el valor predicho disminuirá, y en una cantidad considerable, puesto que el valor absoluto del coeficiente es alto. La cuarta característica también es, por consiguiente, importante.

Los coeficientes de regresión lineal se almacenan en el atributo `.coef_` del modelo entrenado:

```
feature_weights = model.coef_
```

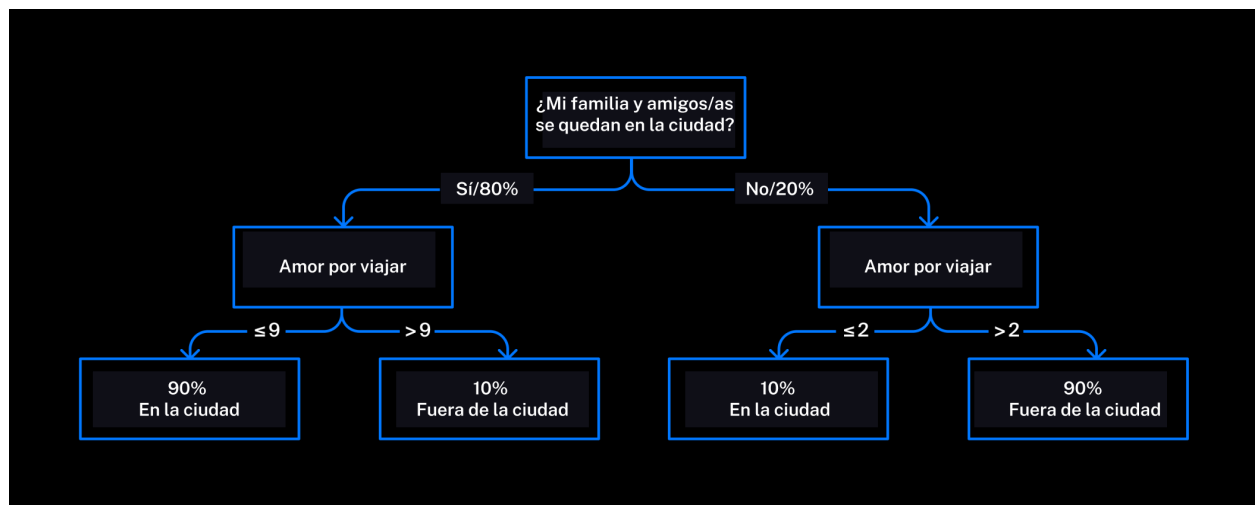
Aquí almacenamos los coeficientes de todas las características en la variable `feature_weights`.

Para mostrar el coeficiente nulo (el valor de la predicción cuando los valores de todas las características son 0), utiliza el atributo `.intercept_`:

```
weight_0 = model.intercept_
```

¡Genial! Hemos averiguado cosas de los modelos lineales: necesitamos mirar los valores absolutos de los coeficientes. ¿Pero que hay de los árboles? Si hay un árbol, normalmente utilizamos el parámetro que más considerablemente predetermina las predicciones en los nodos subsecuentes. La importancia de la característica se define por rango en lugar de coeficientes.

Un importante matiz: cuanto más cerca esté una característica de la base del árbol (la parte superior, en la imagen de abajo), mayor será su importancia e impacto en los resultados previstos.



La importancia de la característica para los árboles (para algoritmos de regresión y clasificación) se almacena en el atributo `.feature_importances_` del modelo entrenado:

```
importances = model.feature_importances_
```

Determinar la importancia de la característica no es la tarea más obvia cuando tenemos un solo árbol. Cuando tenemos conjuntos de modelos (potenciación del gradiente y bosque aleatorio), es todo un campo de estudio para el cual se desarrollan algoritmos especiales. Pero en la vida real, probablemente no profundizarás tanto en los aspectos teóricos de la importancia informática. Las implementaciones del bosque aleatorio y la potenciación del gradiente en sklearn también almacenan la importancia de la característica en el atributo `.feature_importances_`.